



# Agentic Flow Overview

The project implements a sophisticated fraud detection pipeline using **LangGraph**. The workflow is structured as a directed graph where specialized "agents" (nodes) perform tasks, and an LLM-driven **Decision Engine** (powered by Ollama) manages transitions and self-correction loops.

## Workflow Visualization

```
graph TD
    START((START)) --> IngestionAgent[Ingestion Agent]
    IngestionAgent --> |Retry: Noise/Drift| IngestionAgent
    IngestionAgent --> |Success| BalanceAgent[Balance Agent]
    BalanceAgent --> |Retry: Low Quality| BalanceAgent
    BalanceAgent --> |Success| StrategyAgent[Strategy Agent]
    StrategyAgent --> |Retry: Not Robust| StrategyAgent
    StrategyAgent --> |Success| EvaluationAgent[Evaluation Agent]
    EvaluationAgent --> |Correction Order| BalanceAgent
    EvaluationAgent --> |Correction Order| StrategyAgent
    EvaluationAgent --> |Success: Deploy| COMPLETE((COMPLETE))
```

## Agent Roles & Toolkits

### 1. Ingestion Agent

**Objective:** Scrape, label, and profile the latest Reddit fraud data.

- **Tools:** `reddit_scraper`, `reddit_labeller`, `post_processor`, `dataset_append`, `dataset_profiler`, `label_review`.
- **Decision Logic:** Checks for `LABEL_NOISE_THRESHOLD`, `SLANG_DRIFT_THRESHOLD`, and scraper return codes. If the label quality is low or distribution drift is too high, it triggers a retry.

### 2. Balance Agent

**Objective:** Resolve class imbalance using synthetic data generation.

- **Tools:** `balance_search` (optimal ratio), `ctgan_runner` (synthetic gen), `synthetic_quality` (JSD check).
- **Decision Logic:** Evaluates the **Jensen-Shannon Divergence (JSD)**. If the synthetic data

distribution deviates too much from reality ( `MIN_JS_DIVERGENCE_ACCEPT` ), it retries balancing with different parameters.

### 3. Strategy Agent

**Objective:** Enhance model robustness via adversarial training.

- **Tools:** `adversarial_trainer` , `attack_surface` .
- **Decision Logic:** Runs **Adv-CTGAN** or similar adversarial scripts. It measures the "Robustness Gain" (Adversarial F1 - Baseline F1). If no gain is recorded, it adjusts the focus (e.g., long-form text vs. transactional) and retries.

### 4. Evaluation Agent

**Objective:** Final validation before "deployment".

- **Tools:** `evaluation_runner` .
- **Decision Logic:** Enforces strict performance gates:
  - **F1 Score**  $\geq$  `TARGET_F1_THRESHOLD`
  - **Robustness Score**  $\geq$  `TARGET_ROBUSTNESS_THRESHOLD`
- **Feedback Loop:** If thresholds aren't met, it issues a **Correction Order**, routing the workflow back to either the Balance Agent (for utility issues) or Strategy Agent (for robustness issues).

## Decision Engine

Each agent node invokes the [DecisionEngine](file:///home/norm/Projects/Reddit-dataset/agent-new/graph.py#58-100), which:

1. Standardizes the tool outputs into a JSON payload.
2. Queries an LLM (Ollama `qwen2.5:7b` ) to reason about the state.
3. Returns a structured decision: `summary` , `action` , `confidence` , and `metadata` .
4. Provides a rule-based fallback if the LLM or Ollama service is unavailable.